

GoldenSetAuditor

Interview Defense Guide — v0.2.0

Evaluation dataset quality auditor for LLM / RAG applications

1. What Is GoldenSetAuditor?

GoldenSetAuditor audits golden evaluation datasets for LLM and RAG applications **before benchmark scores are trusted**. It accepts a pandas DataFrame and a GoldenSetAuditConfig, runs seven structured checks, and returns a GoldenSetReport with per-finding PASS / WARN / FAIL / INSUFFICIENT_INPUT status.

It is an **auditor, not a scorer**. It does not evaluate model answers. Every finding includes an explicit truth boundary: the tool flags suspicious patterns; the evaluation lead confirms whether an example should be corrected, removed, or kept.

The seven checks:

Check	What it detects
Conflicting duplicate scan	Same prompt, different expected answers — label conflict (FAIL)
Exact duplicate scan	Same prompt, same expected answer — redundant example (WARN)
Weak expected answer	Expected answer with too few meaningful tokens (WARN)
Ambiguous prompt heuristic	Anaphoric refs or multiple questions in one prompt (WARN)
Over-easy example	High token Jaccard between prompt and expected answer (WARN)
Near-duplicate scan	Paraphrased prompts with high pairwise token similarity (WARN)
Category coverage	Categories with fewer examples than configured minimum (WARN)

2. Algorithmic Implementations

2.1 Token Jaccard Similarity

The core similarity primitive used by near-duplicate, over-easy, and ambiguous prompt checks. Text is tokenized by extracting alphanumeric words (regex `[a-zA-Z0-9]+`), lowercased, and filtered against a stopword list (closed-class words like 'the', 'a', 'is', 'and'). Jaccard similarity = $|A \cap B| / |A \cup B|$ over the resulting token sets. Stopword removal improves signal quality: comparing 'What is the PTO policy?' to 'What is the PTO rule?' without stopword removal gives a falsely low Jaccard because 'the' dominates the union.

2.2 Conflicting Duplicate Scan

Groups rows by normalized input text (stripped, lowercased). For any group with more than one row, checks whether all expected answers are identical. If answers differ, status is FAIL (label conflict); if all answers match, status is WARN (redundant duplicate). FAIL because conflicting expected answers are a structural data integrity problem — the ground truth is undefined for that prompt.

2.3 Weak Expected Answer

Counts meaningful tokens (post-stopword-filter) in each expected answer. If the count is below `min_expected_words` (default 4), the row is flagged WARN. A one-word expected answer like 'Yes.' cannot distinguish a model that answers correctly from one that retrieves any affirmative sentence.

2.4 Ambiguous Prompt Heuristic

Applies two heuristics: (a) presence of anaphoric reference patterns (regex for 'it', 'this', 'that', 'they', 'there') in short prompts (< 7 meaningful tokens), and (b) multiple question marks or 'and' combined with at least one question mark. Returns WARN when either condition fires. These are heuristics, not proofs — the evaluator must confirm whether the prompt is genuinely ambiguous.

2.5 Over-easy Example

Computes token Jaccard between the prompt and the expected answer for each row. If the overlap $\geq$ `overeasy_overlap_threshold` (default 0.72), the row is flagged WARN. High overlap suggests the answer is essentially contained in the prompt — a model can score well by extracting tokens from the input without understanding.

2.6 Near-Duplicate Pairwise Scan

Iterates over all pairs of distinct input texts ($O(n^2)$). For each pair with token Jaccard $\geq$ `near_duplicate_threshold` (default 0.82), emits a WARN finding. The scan is skipped and returns `INSUFFICIENT_INPUT` when the number of pairs exceeds `max_pairwise_checks` (default 25,000) to avoid compute overrun on large datasets.

2.7 Category Coverage

Groups rows by `category_col`. Any category with fewer than `min_category_count` (default 3) examples is flagged WARN. Returns `INSUFFICIENT_INPUT` when no `category_col` is configured. A category with 1-2 examples produces unreliable per-category accuracy estimates.

3. Status Rollup

The overall report status is the worst finding: **FAIL > WARN > INSUFFICIENT_INPUT > PASS**. Only conflicting duplicate labels produce FAIL — that is a structural violation. All other findings are WARN (suspicious but requires confirmation) or INSUFFICIENT_INPUT (check skipped due to missing config).

Status	Meaning	Action
PASS	No suspicious patterns detected	Proceed; document the audit
WARN	Suspicious example — review required	Evaluator confirms whether to keep, fix, or remove
FAIL	Label conflict — same prompt, different answers	Resolve before trusting any benchmark score
INSUFFICIENT_INPUT	Check skipped — missing config	Provide config to enable the check

4. Design Decisions

4.1 Conflicting labels → FAIL, everything else → WARN

A prompt with conflicting expected answers has undefined ground truth. There is no evaluation interpretation that resolves this — the label is structurally broken. All other findings are statistical patterns requiring domain judgment: a near-duplicate might be an intentional paraphrase test case; a weak expected answer might be valid for a classification task.

4.2 Stopword filtering in Jaccard

Without stopwords removal, short prompts that share only common function words ('what', 'is', 'the') score as more similar than they are. Filtering to content words makes Jaccard a signal for semantic proximity rather than syntactic pattern matching.

4.3 Configurable pairwise cap

`max_pairwise_checks` prevents the near-duplicate scan from running $O(n^2)$ pairs on large datasets. Returning `INSUFFICIENT_INPUT` with a recommendation to run on stratified batches is safer than silently skipping or crashing.

4.4 Truth boundary in every report

The HTML and Markdown outputs include an explicit interpretation note: 'This report audits golden-set quality. It does not evaluate model answers.' This prevents the report from being misread as a model evaluation.

4.5 `id_col` for traceable findings

When `id_col` is configured, findings reference rows by the application's native identifier (e.g., 'Q42') rather than pandas integer index. This makes findings actionable without requiring the reviewer to cross-reference row positions.

5. Hard Interview Questions

Q: Token Jaccard is order-insensitive. Could two completely different prompts with the same vocabulary score as near-duplicates?

A: Yes — two prompts sharing many content words but in different arrangements (e.g., asking the inverse question) could score high on Jaccard. This is a known limitation. The check returns `WARN`, not `FAIL`, precisely because human confirmation is required. The recommendation explicitly says 'deduplicate or mark as intentional paraphrase'. An n-gram overlap or sentence embedding approach would be more semantically precise but would add a model dependency — that is a documented roadmap extension.

Q: Why is the pairwise scan $O(n^2)$? Wouldn't approximate nearest-neighbor (ANN) be better?

A: For golden sets, n is typically small (hundreds to a few thousand examples). The $O(n^2)$ exact scan is simple, zero-dependency, and tractable for those sizes. The `max_pairwise_checks` cap ensures it fails fast if someone applies the tool to a dataset too large for brute-force comparison. ANN (e.g., FAISS, Annoy) is a correct next step for large evaluation suites — it is documented as a roadmap item.

Q: The ambiguous prompt heuristic looks very simple. What does it miss?

A: It misses ellipsis without anaphora ('Summarize it.' passes if 'it' is absent), presupposition failures ('When did you stop lying?' — false premise), and context-dependent ambiguity ('What is the policy?' where 'policy' is ambiguous without a prior turn). These require deeper linguistic or discourse analysis. The heuristic catches the most common surface pattern at zero model cost. The tool is explicitly scoped as a v0 heuristic auditor, not a full NLP pipeline.

Q: How does this handle multi-turn or context-dependent evaluation examples?

A: The current tool assumes single-turn question/answer pairs. Multi-turn conversation auditing is not supported in v0 — the checks operate on single `input_col` and `expected_col` values. A multi-turn extension would require conversation-level deduplication and context-aware ambiguity detection. That is scoped as a future capability.

Q: Can a golden set that passes this audit still produce misleading benchmark scores?

A: Absolutely, and the tool says so explicitly in every report. This auditor does not check whether expected answers are factually correct, whether they represent the agreed-upon business answer, whether the evaluation metric (exact match, ROUGE, LLM judge) is appropriate for the task, or whether the golden set covers the actual production query distribution. It catches common structural quality problems, not all validity threats.

Q: What does `INSUFFICIENT_INPUT` mean for the near-duplicate scan?

A: When $n*(n-1)/2$ pairs exceed max_pairwise_checks, the scan cannot run without risking excessive compute time. INSUFFICIENT_INPUT signals that the check was skipped — not that the dataset is clean. The recommendation is to increase the cap or run on batches. This is safer than silently returning PASS on a dataset that was never checked.

Q: Why does the over-easy check use Jaccard between prompt and answer? What about exact-match substring?

A: Token Jaccard is more robust than exact-match substring: it handles word order variation and partial overlap without being fooled by one long unique token. A prompt containing a verbatim answer substring would score extremely high on Jaccard and be caught. But a prompt that contains all answer words in scrambled order (unusual) would also be flagged. The WARN-not-FAIL design means these edge cases are reviewed rather than automatically rejected.

6. Truth Boundary

GoldenSetAuditor does not:

- Evaluate model answers or outputs — it audits the dataset only
- Prove that a flagged example will cause incorrect benchmark scores
- Measure inter-annotator agreement or annotation quality
- Replace domain expert review of expected answers
- Detect factual errors, outdated ground truth, or metric misalignment

GoldenSetAuditor does:

- Systematically check seven known golden-set quality failure modes
- Return structured, machine-parseable findings with row-level evidence
- Make the evaluation dataset review step documentable and repeatable
- Distinguish structural violations (FAIL) from suspicious patterns (WARN)

7. Files Index

File	Purpose
src/goldensetauditor/core.py	Main audit logic — 7 checks, GoldenSetAuditConfig, GoldenSetReport
tests/test_goldensetauditor.py	13 unit tests across 4 test classes
scripts/generate_demo_reports.py	Demo script — generates JSON/MD/HTML outputs
data/demo_golden_set.csv	Demo dataset with synthetic quality issues
README.md	Badges, arch diagram, About, check table, quickstart
CHANGELOG.md	v0.1.0 and v0.2.0 release notes
METHODOLOGY.md	Full check derivations and design rationale
pyproject.toml	Package config — version 0.2.0
.github/workflows/ci.yml	Python 3.10/3.11/3.12 CI matrix
.github/workflows/publish.yml	OIDC PyPI trusted publisher on release

Resume-safe claim: Built **GoldenSetAuditor**, an evaluation dataset quality auditor for LLM/RAG applications that checks golden sets for conflicting expected answers, exact and near-duplicate prompts, weak reference answers, ambiguous questions, over-easy examples, and category

coverage gaps, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.